# Lab Practice Problem 2

Filename: `lab2.c`

In this activity, you will complete a partially implemented C program that reads a list of fruit names from a file, stores them in a dynamically allocated array of strings, and answers a sequence of search queries. This exercise is intended to reinforce your skills in dynamic memory allocation and string handling.

## Try It Out

Write a program that reads an integer `N` from an input file, dynamically allocates an array of `N` strings, and populates the array with `N` fruit names read from the file. After building the list, the program reads an integer `Q` representing the number of queries, then reads `Q` fruit names and prints the index where each fruit appears in the list. If a queried fruit is not found, print `-1`.

**Assumptions**

You may assume each fruit name contains no whitespace and is at most 100 characters long. The input file is correctly formatted and contains valid integer counts. Finally, a fruit appears at most once in the initial list.

Your task is to complete the given C program. You can access the starter code here. The following function prototypes describe the functions you will implement:

`char **create_and_populate_list(FILE *ifile, int *size)`
: Reads the number of fruits from `ifile` and stores it in `*size`. Then dynamically allocates an array of `*size` pointers-to-`char`. For each fruit name read from the file, dynamically allocates just enough space to store the string (including the null terminator) and copies the fruit name into the allocated memory. Returns the resulting dynamically allocated array of strings.

`void destroy_list(char **list, int size)`
: Frees all dynamically allocated memory associated with `list`. This includes freeing each dynamically allocated string stored in the array, followed by freeing the array itself.

`int index_of(char **list, int size, char *query)`
: Searches the array `list` of length `size` for a string that matches `query` exactly (case-sensitive). If found, returns the index where the match occurs. If the string is not found, returns `-1`.

`void search_list(FILE *ifile, char **list, int size)`
: Reads an integer `Q` from `ifile` representing the number of search queries. Then reads `Q` query strings and, for each query, prints one line in the format: `query: index`

**Important** You must not modify any other parts of the program. Your only task is to complete the definitions of the four functions listed above. Do not add new helper functions.

## Input

The program reads input from a file named `fruits.txt`. The file is structured as follows. The first line contains an integer `N` ($1 \leq N \leq 1{,}000$), representing the number of fruits in the list. The next `N` lines each contain exactly one fruit name (one fruit per line, with no whitespace in the name).

After the fruit list, the file contains an integer `Q` ($0 \leq Q \leq 10$) indicating the number of search queries. The following `Q` lines each contain a single fruit name to be searched for in the list.

## Output

For each query fruit name, print one line using the following format: `query: index` where `index` is the 0-based position of `query` in the fruit list, or `-1` if the fruit is not present.

**`fruits.txt`**

```
3
apple
banana
cherry
5
apple
 cherry
watermelon
Banana
banana
```

## Sample Output

```
apple: 0
cherry: 2
watermelon: -1
Banana: -1
banana: 1
```

**Guide Questions**

1. What is the behavior of the program when `Q` is `0`?
2. What happens if memory is not dynamically allocated for each element of the array of strings before calling `strcpy()`?
3. What would happen if `destroy_list()` only freed the array of pointers but did not free each individual string?
4. If the program were extended to support adding new fruits to the list or removing existing ones, what changes would be required in the data structure and memory management strategy?
5. What does `remove_fruit()` assume when removing a string? How would the design change if the string had to be returned to the caller instead of being freed inside the function?

### Sample Solution

The source code can be accessed here.